6.5 Outdated Documentation

There are few things worse than outdated documentation. If you've ever had the problem in an application that you couldn't rely on the documentation (for example, the comment block for a function), you'll invest more time in research in future and, if in doubt, take an additional look at the code. The implementation of an extension will therefore take more time.

How does outdated documentation arise? Usually, this doesn't happen because malicious people want to lead others up the garden path, but mostly it's due to time pressure or because the documentation update task was simply forgotten. What remains, however, is the loss of trust, and thus the documentation loses considerable value.

If you work on updating a function yourself, you can get help from AI tools and have the documentation updated automatically. The `fetchUserData` function serves as an example of such a workflow. It was originally implemented in such a way that you pass it the ID of a user and a callback function. The function itself loads the data from a web server and then calls the callback function. The function is now implemented as an `async` function thanks to a modification. It only accepts the user's ID and returns a `Promise` object with the loaded user data. However, the function documentation still refers to the old implementation. In the code it looks as follows:

```
/**
 * Fetches user data from the server using the Fetch
API.
 *
 * This function sends a GET request to the server to
retrieve
 * user data based on the provided user ID. The
response is
 * expected to be in JSON format.
 *
```

```
 * @param {number} userId The ID of the user whose
data
 *                         is to be fetched.
 * @param {function} callback A callback function that
will
 *                           be called with the user
data.
 *                           The callback should
accept two
 *                           parameters: an error (if
any) or
 *                           null if the request was
successful
 *                           and the user data.
 *
 * @return {void}
 */
async function fetchUserData(userId: number) {
  const response = await
fetch(`/api/users/${userId}`);
  if (!response.ok) {
    throw new Error('Failed to fetch user data');
  }
  return response.json();
}
```

The description of the function is still correct, but the explanation of the parameters and the return value is incorrect. To resolve this issue, you can, for example, use VS Code's GitHub Copilot integration again and submit the following prompt:

Prompt: Please update the documentation for the `fetchUserData` function.

> ```
> /**
>  * Fetches user data from the server.
>  *
>  * @param userId - The ID of the user to fetch data
> for.
>  * @returns A promise that resolves to the user data.
>  * @throws An error if the request fails.
>  */
> ```

```
async function fetchUserData(userId: number) {
  const response = await
fetch(`/api/users/${userId}`);
  if (!response.ok) {
    throw new Error("Failed to fetch user data");
  }
  return response.json();
}
```

What is immediately noticeable here is that Copilot has intervened in the structure of the commentary. The original, detailed description of the function has been removed, and the parameter types have also been omitted. Overall, the result is correct, but some information has been lost. This is a good example of how an AI tool with a prompt that is too simple doesn't necessarily improve the situation. You can modify the prompt so that the result is more like the original variant. However, this is only worthwhile for more extensive source code, as in this case, you would invest so much time in the prompt that you might as well write the documentation yourself.

An alternative is to use another tool such as ChatGPT. The result looks different with the same prompt:

Prompt: Please update the documentation for the following function:

[code of the function]

> Here's the updated documentation for the `fetchUserData` function:
>
> ```
> /**
>  * Fetches user data from the server using the Fetch
> API.
>  *
>  * This asynchronous function sends a GET request to
> the server
>  * to retrieve user data based on the provided user
> ID. The
>  * response is expected to be in JSON format and is
> returned as
>  * a Promise that resolves to the user data.
> ```

```
 *
 * @param {number} userId - The ID of the user whose
data is to
 *                          be fetched.
 *
 * @throws {Error} Will throw an error if the network
request
 *                 fails or if the server returns a
non-OK
 *                 status.
 *
 * @returns {Promise<Object>} A Promise that resolves
to the
 *                           user data object when
the request
 *                           is successful.
 *
 * @example
 * fetchUserData(1)
 *   .then(userData => console.log(userData))
 *   .catch(error => console.error('Error fetching
user data:',
 *                   error));
 */
async function fetchUserData(userId: number) {
  const response = await
fetch(`/api/users/${userId}`);
  if (!response.ok) {
    throw new Error('Failed to fetch user data');
  }
  return response.json();
}
```

[key updates: more details]

ChatGPT hasn't deleted any information but rather has adapted the documentation of the parameters and added an example of how to use the function. You'll also receive a list of the changes made, including the associated explanations.

When using AI tools, you also have to weigh the convenience offered by a tool such as GitHub Copilot, which allows you to modify the code directly in

the development environment, against the quality of another tool, which may require you to paste the source code into another command prompt.